

Foundry: an autonomous model-distillation agent on a single AMD Radeon

AMD AI DevMaster Hackathon — Track 2

abidedavana

Contents

Abstract . . . 3

1. The problem and the scenario . . . 3

2. Why this cannot be an API wrapper . . . 3

3. System . . . 4

    3.1 The agent . . . 4

    3.2 The pipeline (all stages on one GPU) . . . 4

    3.3 One codebase, two backends . . . 4

    3.4 Interface . . . 5

4. Results (granted Radeon: gfx1100 48 GB, ROCm 7.2.4) . . . 5

    4.1 Full-scale build (support-ticket triage; 900 train / 100 holdout) . . . 5

    4.2 Quantization ladder (accuracy measured through llama-server) . . . 6

    4.3 The improvement loop (measured end to end) . . . 6

    4.4 Generality . . . 7

    4.5 What the interaction looks like . . . 7

5. ROCm engineering notes . . . 7

6. Limitations, honestly . . . 7

7. Relation to prior work . . . 8

8. Radeon cloud endpoint deployment — attempted, unavailable . . . 8

9. Reproducing . . . 8

AMD AI DevMaster Hackathon — Track 2 (Agentic AI). Solo entry.

Abstract

Foundry is an AI agent that manufactures small task-specific models on one AMD Radeon GPU. A user describes the model they need in plain language; the agent plans the pipeline, generates synthetic training data with a local 30B-parameter teacher grounded in the user's seed documents, LoRA-fine-tunes a 0.6B student, evaluates it against the teacher's held-out labels, diagnoses failure classes, regenerates targeted data, retrains, and exports a quantized GGUF ready to deploy anywhere. On the contest-granted Radeon (gfx1100, 48 GB, ROCm 7.2.4), the complete loop runs in ~21 minutes and produces a student that agrees with its teacher on **97% of held-out examples** while running **2.6× faster at 1/42 the size** — and against ground-truth labels (a blind re-annotation of the holdout against the policy text), the student matches its 30B teacher at **0.95 exact, 1.00 on unambiguous cases**. Every number in this report was measured on that hardware; the artifacts (datasets, eval reports, benchmark JSON, charts, annotation rationales) ship with the repository.

1. The problem and the scenario

Everyone wants small, private, task-specific models: a triage classifier, an expense router, an extractor — models that run on a laptop, an edge box, or a modest server, with no per-token bill and no data leaving the machine. What stands in the way is not training hardware; it is that the pipeline between "I need a model that does X" and a deployable artifact — dataset design, synthetic generation, fine-tuning, evaluation, error analysis, quantization — is expert work.

Foundry turns that pipeline into a conversation. The user's only inputs are a natural-language description and the seed documents (a policy, a handbook, a spec) that define ground truth. The output is a model file, not chat answers.

2. Why this cannot be an API wrapper

- **Fine-tuning cannot happen over an inference API.** The product's core operation — producing new weights — requires local training.
- **Synthetic data generation is where iteration cost lives.** A single 1,000-example build passes roughly two million tokens through the teacher (policy context plus generations), and the agent's value is iterating — regeneration rounds, previews, larger corpora scale that linearly. Hosted, every loop has a bill attached; on the Radeon the marginal cost is zero, so the agent can afford to iterate freely.
- **Privacy is structural.** The seed documents that ground the labels never leave the machine, and neither does the data derived from them.

Track 2's rule that "core inference processes shall be executed locally on AMD Radeon GPU" is, for Foundry, simply a description of the product.

3. System

3.1 The agent

The agent's brain is the same local model that serves as the teacher (Qwen3-30B-A3B-Instruct, Q4_K_M GGUF), served by llama.cpp's llama-server with OpenAI-compatible tool calling. The agent loop is a compact, framework-free Python module the judges can read in one sitting. Its capabilities map directly onto Track 2's definition:

- **Task decomposition.** The agent plans the build, drafts a machine-checkable TaskSpec (output schema, enum values, balance field, generation facets) from the conversation, and sequences pipeline stages, adapting to results (e.g., choosing targeted regeneration after a diagnosis).
- **Tool invocation.** Eleven tools wrap the pipeline: spec management, preview, dataset generation/augmentation, training, evaluation, failure diagnosis, quantization benchmarking, GGUF export, and an experiment registry. Training/evaluation/export run as isolated subprocesses.
- **Retrieval-grounded generation.** Every synthetic example is generated from the user's seed documents. Small corpora ride whole in the teacher's context; past a configurable budget, a per-request chunk retriever (heading-aware splitting, tf-idf scoring against the target class and facets) selects the relevant sections, and each example records which sections grounded it. Validated on a three-document policy corpus where correct labels require rules from different documents in one claim (e.g., a software-addendum rule deciding the routing of an app-store purchase).
- **Memory management.** A persistent experiment registry records every stage's parameters, metrics, and artifacts. The agent lists and resumes builds across sessions ("continue yesterday's model"), and evaluation results carry the previous run's scores so before/after comparisons are structural rather than remembered.

3.2 The pipeline (all stages on one GPU)

1. **Datagen.** Facet-controlled prompts (persona, tone, length, situation) pin each example to a target class, so class balance is a property of the code, not model whim. Hard validation (schema, enums, length) plus near-duplicate detection (shingle Jaccard) filter the teacher's output; rejects are kept inspectable. Measured acceptance at full scale: **99.0%**.
2. **Training.** transformers + peft LoRA (bf16, r=16) on a Qwen3-0.6B student. Loss is computed only over labeled target tokens — with a 152K vocabulary this avoids a ~2.5 GB fp32 logits upcast and is why the identical code trains on an 8 GB laptop GPU and the Radeon. Unsloth (official AMD support) was validated as an alternate backend on the same hardware (identical loss/eval) and remains a config flag; peft was kept for its process-cleanliness.
3. **Evaluation.** Base (untrained) and tuned students run the same frozen holdout; every report carries its own before/after delta.
4. **Diagnosis → targeted regeneration.** Per-class recall, confusion pairs, and sample failures; the agent then oversamples weak classes ~2:1 into the training set while the holdout stays frozen.
5. **Export + benchmark.** Merged weights convert to GGUF; a quantization ladder (Q4→Q8) is measured for size, llama-bench speed, and — importantly — accuracy served through llama-server, i.e., the artifact users actually deploy.

3.3 One codebase, two backends

Development ran on an RTX 4060 laptop (CUDA); every submission number comes from the granted Radeon (ROCm). No code forks: GPU access goes through exactly two seams — llama-server's HTTP API (backend-agnostic binaries) and torch's device layer (PyTorch-ROCm exposes HIP as `torch.cuda`). Device selection is a config file.

3.4 Interface

A thin web UI (Flask + one static page) streams the agent's events: every tool call renders with live timing, a telemetry card polls rocm-smi/nvidia-smi (VRAM and utilization visible during every stage), and build-stage badges plus the experiment registry keep the state honest — if the agent claims something it didn't do, the panel shows it. A terminal REPL with the same event stream exists for scripted, reproducible runs.

4. Results (granted Radeon: gfx1100 48 GB, ROCm 7.2.4)

4.1 Full-scale build (support-ticket triage; 900 train / 100 holdout)

Stage	Result	Time
Datagen (30B teacher, 8 slots)	1,000 accepted / 1,010 attempts (99.0%), classes balanced 155–175	17.6 min
LoRA training (171 steps)	final loss 0.0029	2.9 min
Evaluation (n=100)	see below	11 s
Export + served smoke test	610 MB Q8_0, valid JSON	11 s

Model	Category acc	Priority acc	Exact match	JSON valid
Base Qwen3-0.6B	0.38	0.19	0.15	1.00
Foundry student	**0.97**	**0.97**	**0.97**	1.00

The table above measures agreement with the teacher — distillation fidelity. To measure *correctness*, we re-annotated all 100 holdout tickets against the policy text itself, blind to every model output, with per-ticket rationales published in the repository (`policy_labels.jsonl`); 8 tickets were flagged as genuinely ambiguous (dual-issue) with the alternative reading recorded. Against these policy-true labels:

Model	Exact match vs policy (n=100)	On unambiguous tickets (n=92)
Base Qwen3-0.6B	0.14	—
Teacher (30B)	0.95	**1.00**
Foundry student	**0.95**	**1.00**

Every teacher–policy disagreement falls on a pre-flagged ambiguous ticket, where the teacher chose the recorded defensible alternative. The student is not merely copying its teacher: both apply the written policy essentially perfectly, and on every clear-cut case both are flawless. An earlier independent full-scale run scored 0.95 teacher-agreement — the factory reproduces within noise.

Datagen throughput, since it is the optimization target for the teacher workload: 1,000 accepted examples in 17.6 minutes — a sustained **0.95 examples/second** through 8-slot continuous batching on a single card, concurrent with validation and deduplication.

4.2 Quantization ladder (accuracy measured through llama-server)

Model	Size	pp tok/s	tg tok/s	Exact match
Q4_K_M	378 MB	6,753	284.9	0.920
Q5_K_M	**424 MB**	**6,992**	**283.4**	**0.970**
Q6_K	472 MB	7,032	273.9	0.970
Q8_0	610 MB	7,394	256.5	0.970
Teacher (30B Q4_K_M)	17,697 MB	1,400	110.2	(labels)

Q4 costs five points of accuracy; Q5 costs nothing — from Q5_K_M up, served accuracy equals the full-precision checkpoint's on this task. The chosen Q5_K_M student runs **2.6× faster than its teacher in generation, ~5× in prompt processing, at 1/42 the size**.

Two honest notes on that comparison. The tok/s numbers are single-stream llama-bench defaults; and the teacher is a mixture-of-experts model activating only ~3B parameters per token, so raw tok/s understates the efficiency gap — the operative wins are the 42× disk/VRAM footprint, the 5× prompt throughput that dominates batch-classification workloads, and the freedom to run many students concurrently where one teacher barely fits.

The footprint claim, measured on CPUs (llama-bench, no GPU):

Student	Quant	Laptop i7-14650HX	EPYC 9334 (host CPU)
0.6B	Q4_K_M	64.2 tok/s	**89.5 tok/s**
0.6B	Q5_K_M	41.7	82.3
1.7B	Q4_K_M	—	39.2

The manufactured model needs no GPU at all to be useful — on a server CPU the Q4 student reaches ~80% of the 30B teacher's *GPU* generation speed. Q4 is the CPU-deployment pick (its 5-point accuracy cost buys +54% CPU speed); Q5 the accuracy-first pick.

Student right-sizing, measured. We repeated the entire factory run with a 1.7B student: 0.95 exact match — the same accuracy band as the 0.6B (0.95–0.97 across runs) at 3× the size and ~25% lower GPU throughput (Q5: 1,200 MB / 213 tok/s vs 424 MB / 283 tok/s). This task saturates at 0.6B, and that is the point: model size is a measured decision in Foundry, not a guess — the same harness that produced these tables answers "how small can this task go?" for any spec.

4.3 The improvement loop (measured end to end)

On a deliberately small build (59 train examples), evaluation flagged `feature_request` at 0.667 recall. Across targeted augmentation rounds focused on the weak class, the agent grew the training set to 141 examples — the holdout frozen throughout — retrained, and re-evaluated: **exact match 0.667 → 0.833, category accuracy 0.917 → 1.00** on the identical holdout. The loop also behaves honestly when improvement doesn't materialize: in one rehearsal where an augmentation round produced no gain, the agent reported "no measurable gain" with a plausible cause, rather than inflating.

4.4 Generality

From a never-seen expense-approval policy and one conversation, the agent drafted its own spec (different fields: routing_decision × spending_category), built the dataset, trained, evaluated, and exported — **zero code changes**. Harder task (amount thresholds, precedence rules), honest result at small scale: 0.417 exact vs 0.167 base, category field at 0.917 — the kind of gap the improvement loop and larger builds exist to close.

4.5 What the interaction looks like

Abridged from a real session transcript (July 17, dev GPU; the unabridged transcript and all artifacts are in the repository):

> **User:** Yesterday we built a Northbeam ticket triage model and feature_request was the weak class. Find that build and resume it.
>
> → *list_experiments* · → *resume_build* — "I have resumed the build. Dataset: 111 train examples. Last eval: category 91.7%, priority 66.7%, exact 66.7%."
>
> **User:** Diagnose what's weak, then add 30 focused training examples on the weak classes, retrain, and re-evaluate on the same holdout. Report the before/after comparison.
>
> → *diagnose_failures* (feature_request recall 0.667; confusion with technical_issue) → *augment_dataset(n=30, focus=[feature_request])* (train 111 → 141, holdout untouched) → *train_student* → *evaluate_student* — "Category accuracy 91.7% → **100%**, exact match 66.7% → **83.3%** on the same holdout."

One compound instruction; four tools chained autonomously; a before/after answer grounded in tool results. This arc — resume from memory, diagnose, targeted regeneration, honest comparison — is the product's core interaction and the demo video's centerpiece.

5. ROCm engineering notes

- Environment: ROCm 7.2.4, torch 2.10.0+rocm, llama.cpp b9766 (HIP), peft 0.19 / transformers 5.5 — the granted image's stack, used as shipped, documented in the repo's notes/.
- Three distinct GPU workloads timeshare one card: MoE teacher serving (batched generation, 8 parallel slots), LoRA training, and student serving. On 48 GB they coexist; on the 8 GB dev GPU the agent orchestrates teacher stop/start around training — same code, config flag.
- The masked-loss trick (§3.2) and batch-size discipline came from the 8 GB device and transfer as pure headroom on the Radeon.
- Unsloth's AMD backend (Triton-ROCm kernels) was gate-tested on this hardware: trains correctly (loss parity with peft); its global transformers patching motivated our subprocess isolation design.
- llama.cpp tool-calling (--jinja) with Qwen3 models worked unmodified on ROCm — the entire agent runs on AMD silicon.

6. Limitations, honestly

- The current TaskSpec targets classification/structured-output tasks (enum and string fields, exact-match scoring). Free-form generation tasks would need judged evaluation — the architecture has the seam (teacher-as-judge) but it is not built.

- Student quality is bounded by teacher labels. We measured that bound for the triage task (§4.1): the teacher is policy-perfect on unambiguous cases, so the ceiling cost was zero here — but a noisier domain or a weaker teacher would transfer its errors into the student, and the policy-true audit (blind re-annotation, published rationales) is the check we'd run each time. The annotator was the project author, blind to model outputs; the labels and rationales are in the repo for independent audit.
- The agent's brain, when run with a small stand-in model during development, occasionally narrated actions without invoking tools; system-prompt rules plus a UI that displays real state mitigate, and the 30B brain is markedly more reliable.
- Priority distribution in synthetic data required explicit situation-facet steering to avoid teacher bias toward dramatic cases; the mechanism is general but each new domain deserves a glance at its distribution.

7. Relation to prior work

Kiln offers a local-first *workbench* where a human drives fine-tuning stage by stage (training typically exported to cloud services); flag autotuners (llama-optimus, TurboLLM) tune serving parameters. Foundry differs in kind: an autonomous closed loop — plan → generate → train → evaluate → diagnose → regenerate → export — running entirely on local AMD hardware, with the model artifact as the product. AMD's own technical article on fine-tuning with Unsloth-generated synthetic data validates the workflow Foundry automates; amd/gaia issue #667 requests exactly this capability for AMD's agent ecosystem.

8. Radeon cloud endpoint deployment — attempted, unavailable

We attempted to publish the distilled, quantized student as a dedicated endpoint on the Radeon cloud platform (its documented vLLM-based serving path). During the contest window the portal's **Model APIs category contained no deployable templates** — "No templates in this category yet" (screenshot: `evidence/model-apis-empty-2026-07-28.png`) — so endpoint deployment was not available to participants in practice.

On the substance of the criterion (quantization/distillation with core inference on Radeon cloud): that is this entire system. Teacher serving, LoRA training, and quantized-student serving — every core inference process — ran on the granted Radeon cloud instances, and both distillation and quantization are the product itself.

9. Reproducing

```
# Radeon (granted instance image):
pip install requests sentencepiece protobuf matplotlib flask /opt/llama.cpp/gguf-py
hf download unsloth/Qwen3-30B-A3B-Instruct-2507-GGUF Qwen3-30B-A3B-Instruct-2507-Q4_K_M.gguf
--local-dir models/teacher
hf download Qwen/Qwen3-0.6B --local-dir models/student/Qwen3-0.6B
python run_slice.py --config configs/cloud_radeon.toml          # full build
python run_bench.py --config configs/cloud_radeon.toml --run-dir runs/<run>  # ladder
python serve_ui.py  --config configs/cloud_radeon.toml          # the agent UI
```

Hardware used: AMD Radeon (gfx1100, 48 GB VRAM) on the contest cloud; AMD EPYC 9334 host. Dev machine: RTX 4060 Laptop 8 GB (no submission numbers). All run artifacts referenced here are committed under `runs/`.